

DSA Patterns & Templates

01

In This Edition

1	Overview	2
	1.1 How to Use This File	2
	1.2 Universal Problem-Solving Workflow	2
2	Pattern Recognition Cheat Table	2
3	The Patterns	3
	3.1 Two Pointers	3
	3.2 Sliding Window (Fixed + Variable)	4
	3.3 Fast & Slow Pointers (Cycle Detection)	6
	3.4 Merge Intervals	7
	3.5 Cyclic Sort	8
	3.6 In-place Linked List Reversal	9
	3.7 BFS (Tree + Grid + Graph)	10
	3.8 DFS (Tree + Grid + Graph)	12
	3.9 Backtracking	13
	3.10 Subsets / Permutations / Combinations	14
	3.11 Binary Search (+ On Answer / Monotonic Predicate)	15
	3.12 Top-K / Heap	17
	3.13 K-way Merge	17
	3.14 Two Heaps (Median of Stream)	18
	3.15 Monotonic Stack	19
	3.16 Prefix Sum	20
	3.17 Union-Find (DSU)	21
	3.18 Topological Sort	22
	3.19 Trie	23
	3.20 Dynamic Programming	24
	3.21 Greedy	28
	3.22 Bit Manipulation	29
4	Complexity Quick Reference	30
	4.1 Big-O of Common Data Structure Operations	30
	4.2 Common Algorithm Complexities	31
	4.3 Acceptable Complexity vs Input Size N	31
5	Interview Communication Tips	31
	5.1 Before You Code	32
	5.2 While Coding	32
	5.3 After Coding	32
	5.4 Recovery Tactics	32
6	Typical Mistakes Candidates Make	32
7	Revision Cheat Sheet	33
	7.1 The 20 Must-Know Patterns	33
	7.2 Decision Tree (30-second pattern picker)	33
	7.3 Quick Complexity Reminder	34

The "when you see X → use Y" trigger guide for FAANG coding rounds. Dense, copy-pasteable, built for fast recognition and last-minute revision.

1 Overview

1.1 How to Use This File

- › **Before interview:** read the Pattern Recognition Cheat Table (§2) until you can map a problem sentence to a pattern in under 10 seconds.
- › **During prep:** for each pattern in §3, memorize the trigger signal, copy the template, and solve the listed problems.
- › **Night before:** skim the Revision Cheat Sheet (§7).

1.2 Universal Problem-Solving Workflow

```

1 1. CLARIFY    - constraints (N size), edge cases, return type, mutate or not?
2 2. EXAMPLES  - run 2-3 examples by hand, including edge cases
3 3. BRUTE FORCE - state it out loud, give its complexity
4 4. OPTIMIZE  - identify the bottleneck; apply pattern recognition
5 5. CODE      - write clean code with variable names that tell the story
6 6. TEST      - trace through your examples + edge cases on the code
7 7. COMPLEXITY - state time and space, justify each term

```

Rule of thumb: always state brute force before optimizing. Interviewers reward structured thinking over jumping to solutions.

2 Pattern Recognition Cheat Table

The most important section. Map the problem's key phrases to a pattern before writing a single line of code.

If the problem says / asks for ...	Reach for this pattern
Sorted array, find pair/triplet with target sum	Two Pointers
Remove duplicates in-place, sorted array	Two Pointers
Longest/shortest subarray/substring with condition	Sliding Window (variable)
Subarray of exact size K with property	Sliding Window (fixed)
Linked list cycle, middle of list, palindrome LL	Fast & Slow Pointers
Overlapping intervals, merge/insert intervals	Merge Intervals
Array 0..N-1, find missing/duplicate number	Cyclic Sort
Reverse a linked list, K-group reversal	In-place LL Reversal
Shortest path, level-order traversal, minimum steps	BFS
All paths, connected components, flood fill	DFS / Backtracking
All subsets, all permutations, all combinations	Backtracking / Subsets

If the problem says / asks for ...	Reach for this pattern
"Find minimum/maximum satisfying condition", search space is monotonic	Binary Search on Answer
Top K largest/smallest, K closest, Kth element	Heap (Top-K)
Merge K sorted lists/arrays	K-way Merge (Heap)
Median of stream, two halves balanced	Two Heaps
Next greater element, stock prices, daily temperatures	Monotonic Stack
Subarray sum equals K, running sum queries	Prefix Sum
Connected components, union/cycle in graph	Union-Find (DSU)
Course schedule, dependency order, DAG ordering	Topological Sort
Word search, prefix matching, autocomplete	Trie
Count ways, min cost, max value, overlapping sub-problems	Dynamic Programming
0/1 choice per item, weight/capacity constraint	DP – 0/1 Knapsack
Unlimited copies of items allowed	DP – Unbounded Knapsack
Longest common subsequence/substring	DP – LCS
Longest increasing subsequence	DP – LIS
Grid, min path, unique paths	DP – Matrix/Grid
Intervals, burst balloons, matrix chain	DP – Interval DP
Locally optimal choice leads to global optimum	Greedy
XOR, AND, OR, bit shift tricks, unique element	Bit Manipulation

3 The Patterns

3.1 Two Pointers



Figure 1. On sorted input, two pointers from both ends turn an $O(n^2)$ pair-search into $O(n)$: move the pointer that brings the sum toward the target.

Trigger / When to Use

When you see: sorted array/string, find a pair or triplet, remove duplicates in-place, partition array.

Core Idea Use two indices that move toward each other (or in the same direction) to eliminate the need for a nested loop. Works on sorted input or with a logical invariant.

Template

```
1 def two_pointers(arr, target):
2     left, right = 0, len(arr) - 1
3     while left < right:
4         current = arr[left] + arr[right]
5         if current == target:
6             return [left, right]      # found
7         elif current < target:
8             left += 1                  # need larger sum
9         else:
10            right -= 1                  # need smaller sum
11    return []
12
13 # Same-direction variant (remove duplicates)
14 def remove_duplicates(arr):
15     slow = 0
16     for fast in range(1, len(arr)):
17         if arr[fast] != arr[slow]:
18             slow += 1
19             arr[slow] = arr[fast]
20    return slow + 1
```

Complexity: Time $O(N)$, Space $O(1)$

Classic Problems

- › Two Sum II (sorted array) — LeetCode 167
- › 3Sum — LeetCode 15
- › Container With Most Water — LeetCode 11
- › Remove Duplicates from Sorted Array — LeetCode 26

Pitfalls

- › Forgetting to skip duplicate values in 3Sum (leads to duplicate triplets).
- › Using on unsorted input without sorting first (add $O(N \log N)$ sort cost).
- › Off-by-one: $\text{left} < \text{right}$ vs $\text{left} \leq \text{right}$.

3.2 Sliding Window (Fixed + Variable)

Sliding window — fixed window of size k slides in $O(n)$

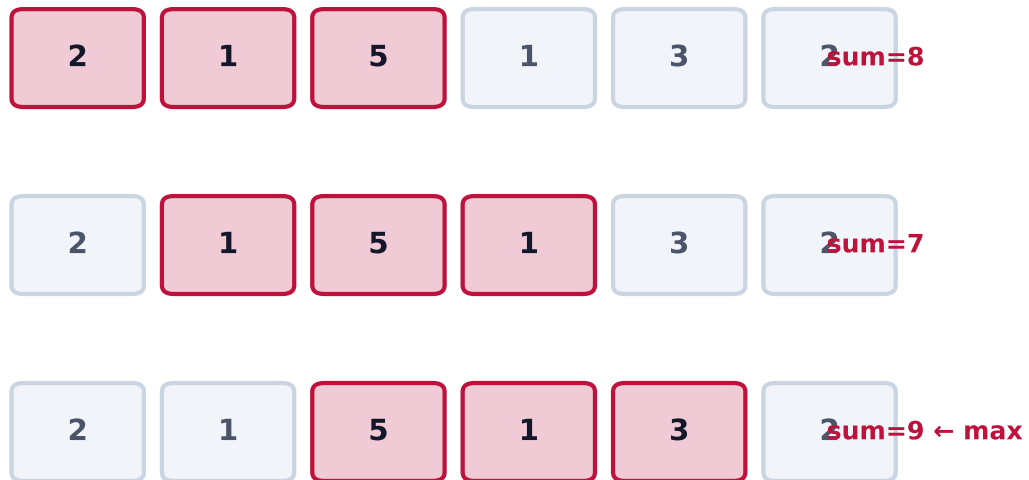


Figure 2. Instead of recomputing each window from scratch ($O(n \cdot k)$), slide the window: add the entering element and drop the leaving one, achieving $O(n)$.

Trigger / When to Use

When you see: contiguous subarray/substring, fixed window of size K, longest/shortest window satisfying a condition.

Core Idea Maintain a window [left, right] over the sequence. Expand right to grow; shrink left when the constraint is violated. Avoids recomputing the full window from scratch each step.

Template — Fixed Window (size K)

```
1 def fixed_window(arr, k):
2     window_sum = sum(arr[:k])
3     max_sum = window_sum
4     for i in range(k, len(arr)):
5         window_sum += arr[i] - arr[i - k] # slide: add right, remove left
6         max_sum = max(max_sum, window_sum)
7     return max_sum
```

Template — Variable Window

```
1 from collections import defaultdict
2
3 def variable_window(s, k):
4     """Longest substring with at most k distinct characters."""
5     char_count = defaultdict(int)
6     left = 0
7     max_len = 0
8     for right in range(len(s)):
9         char_count[s[right]] += 1
10        while len(char_count) > k: # shrink until valid
```

```

11         char_count[s[left]] -= 1
12         if char_count[s[left]] == 0:
13             del char_count[s[left]]
14         left += 1
15         max_len = max(max_len, right - left + 1)
16     return max_len

```

Complexity: Time $O(N)$, Space $O(K)$ for the hash map

Classic Problems

- › Maximum Sum Subarray of Size K
- › Longest Substring Without Repeating Characters — LeetCode 3
- › Minimum Window Substring — LeetCode 76
- › Fruit Into Baskets — LeetCode 904

Pitfalls

- › Shrinking the window with `if` instead of `while` (can leave window invalid).
- › Not updating the result both inside and after the while shrink loop.
- › Variable window: forgetting to delete keys with count 0 (artificially inflates distinct count).

3.3 Fast & Slow Pointers (Cycle Detection)

Trigger / When to Use

When you see: linked list cycle, find middle node, palindrome linked list, detect duplicate in array (Floyd's algorithm).

Core Idea Two pointers move at different speeds (fast = 2 steps, slow = 1 step). If there's a cycle they must meet. Without a cycle, fast reaches the end first.

Template

```

1 class ListNode:
2     def __init__(self, val=0, next=None):
3         self.val = val
4         self.next = next
5
6 def has_cycle(head):
7     slow = fast = head
8     while fast and fast.next:
9         slow = slow.next
10        fast = fast.next.next
11        if slow is fast:
12            return True
13    return False
14
15 def find_cycle_start(head):
16     slow = fast = head
17     while fast and fast.next:
18         slow = slow.next
19         fast = fast.next.next
20         if slow is fast:
21             break
22     else:
23         return None # no cycle

```

```

24     # Reset one pointer to head; advance both at speed 1
25     slow = head
26     while slow is not fast:
27         slow = slow.next
28         fast = fast.next
29     return slow                # cycle start
30
31 def find_middle(head):
32     slow = fast = head
33     while fast and fast.next:
34         slow = slow.next
35         fast = fast.next.next
36     return slow                # middle node (for even length: second middle)

```

Complexity: Time $O(N)$, Space $O(1)$

Classic Problems

- › Linked List Cycle — LeetCode 141
- › Linked List Cycle II (find start) — LeetCode 142
- › Middle of the Linked List — LeetCode 876
- › Find the Duplicate Number — LeetCode 287

Pitfalls

- › Checking `slow == fast` before moving on first iteration (they start equal).
- › Not handling `head is None` or single-node lists.
- › Find Duplicate: the array must be treated as a linked list (`index` → `value` → `next index`).

3.4 Merge Intervals

Trigger / When to Use

When you see: overlapping intervals, schedule conflicts, insert interval, minimum meeting rooms.

Core Idea Sort intervals by start time. Walk through; if the current interval overlaps the last merged one, merge by extending the end. Otherwise, append as new.

Template

```

1 def merge_intervals(intervals):
2     if not intervals:
3         return []
4     intervals.sort(key=lambda x: x[0])
5     merged = [intervals[0]]
6     for start, end in intervals[1:]:
7         if start <= merged[-1][1]:          # overlap: start <= prev end
8             merged[-1][1] = max(merged[-1][1], end)
9         else:
10            merged.append([start, end])
11    return merged
12
13 def insert_interval(intervals, new_interval):
14    result = []
15    i = 0
16    n = len(intervals)

```



```

17 # Add all intervals that end before new_interval starts
18 while i < n and intervals[i][1] < new_interval[0]:
19     result.append(intervals[i]); i += 1
20 # Merge all overlapping
21 while i < n and intervals[i][0] ≤ new_interval[1]:
22     new_interval[0] = min(new_interval[0], intervals[i][0])
23     new_interval[1] = max(new_interval[1], intervals[i][1])
24     i += 1
25 result.append(new_interval)
26 result.extend(intervals[i:])
27 return result

```

Complexity: Time $O(N \log N)$ (sort), Space $O(N)$

Classic Problems

- › Merge Intervals — LeetCode 56
- › Insert Interval — LeetCode 57
- › Meeting Rooms II (min rooms) — LeetCode 253
- › Non-overlapping Intervals — LeetCode 435

Pitfalls

- › Forgetting to sort before merging.
- › Using $<$ instead of $\leq$ for overlap check (intervals that just touch should merge if problem says so).
- › Mutating the original list element; use `merged[-1][1] = max(...)`.

3.5 Cyclic Sort

Trigger / When to Use

When you see: array containing numbers in range $[1, N]$ or $[0, N-1]$, find missing/duplicate/all missing numbers.

Core Idea Each number belongs at index `num - 1`. Walk the array; if `arr[i]` is not at its correct index, swap it there. After one pass, scan for misplaced numbers.

Template

```

1 def cyclic_sort(nums):
2     i = 0
3     while i < len(nums):
4         correct = nums[i] - 1 # where nums[i] should live
5         if nums[i] ≠ nums[correct]: # not in place
6             nums[i], nums[correct] = nums[correct], nums[i]
7         else:
8             i += 1
9     return nums
10
11 def find_missing(nums):
12     cyclic_sort(nums)
13     for i, num in enumerate(nums):
14         if num ≠ i + 1:
15             return i + 1
16     return len(nums) + 1
17

```

```

18 def find_duplicate(nums):
19     i = 0
20     while i < len(nums):
21         correct = nums[i] - 1
22         if nums[i] != i + 1:
23             if nums[i] == nums[correct]: # duplicate found
24                 return nums[i]
25             nums[i], nums[correct] = nums[correct], nums[i]
26         else:
27             i += 1
28     return -1

```

Complexity: Time $O(N)$, Space $O(1)$

Classic Problems

- › Missing Number — LeetCode 268
- › Find All Numbers Disappeared in an Array — LeetCode 448
- › Find the Duplicate Number — LeetCode 287
- › First Missing Positive — LeetCode 41

Pitfalls

- › Infinite loop if you don't guard against `nums[i] == nums[correct]` when finding duplicates.
- › Range $[0, N-1]$ vs $[1, N]$ — adjust correct index accordingly.

3.6 In-place Linked List Reversal

Trigger / When to Use

When you see: reverse a linked list, reverse in groups of K, reverse between positions L and R.

Core Idea Iteratively redirect next pointers using three pointers: `prev`, `curr`, `next_node`. No extra space needed.

Template

```

1 def reverse_list(head):
2     prev = None
3     curr = head
4     while curr:
5         next_node = curr.next
6         curr.next = prev
7         prev = curr
8         curr = next_node
9     return prev
10
11 def reverse_sublist(head, left, right):
12     """Reverse nodes from position left to right (1-indexed)."""
13     dummy = ListNode(0)
14     dummy.next = head
15     prev = dummy
16     for _ in range(left - 1):
17         prev = prev.next
18     curr = prev.next
19     for _ in range(right - left):

```

```

20     next_node = curr.next
21     curr.next = next_node.next
22     next_node.next = prev.next
23     prev.next = next_node
24     return dummy.next
25
26 def reverse_k_groups(head, k):
27     """Reverse every k nodes."""
28     dummy = ListNode(0)
29     dummy.next = head
30     group_prev = dummy
31     while True:
32         kth = get_kth(group_prev, k)
33         if not kth:
34             break
35         group_next = kth.next
36         prev, curr = kth.next, group_prev.next
37         while curr != group_next:
38             tmp = curr.next
39             curr.next = prev
40             prev = curr
41             curr = tmp
42         tmp = group_prev.next
43         group_prev.next = kth
44         group_prev = tmp
45     return dummy.next
46
47 def get_kth(curr, k):
48     while curr and k > 0:
49         curr = curr.next
50         k -= 1
51     return curr

```

Complexity: Time $O(N)$, Space $O(1)$

Classic Problems

- › Reverse Linked List — LeetCode 206
- › Reverse Linked List II — LeetCode 92
- › Reverse Nodes in k-Group — LeetCode 25
- › Reorder List — LeetCode 143

Pitfalls

- › Losing the next pointer before redirecting it.
- › Off-by-one on position indices (1-indexed vs 0-indexed).
- › Not attaching the dummy node's tail to the remaining list after reversal.

3.7 BFS (Tree + Grid + Graph)

Trigger / When to Use

When you see: shortest path in unweighted graph, minimum steps, level-order traversal, nearest X in grid, connected components (layer by layer).

Core Idea Explore nodes level by level using a queue. Guarantees shortest path in unweighted graphs. Track visited set to avoid revisiting.

Template — Tree Level-Order

```

1 from collections import deque
2
3 def level_order(root):
4     if not root:
5         return []
6     result = []
7     queue = deque([root])
8     while queue:
9         level_size = len(queue)
10        level = []
11        for _ in range(level_size):
12            node = queue.popleft()
13            level.append(node.val)
14            if node.left: queue.append(node.left)
15            if node.right: queue.append(node.right)
16        result.append(level)
17    return result

```

Template — Grid BFS

```

1 def bfs_grid(grid, start_r, start_c):
2     rows, cols = len(grid), len(grid[0])
3     queue = deque([(start_r, start_c, 0)]) # (row, col, distance)
4     visited = {(start_r, start_c)}
5     directions = [(0,1),(0,-1),(1,0),(-1,0)]
6     while queue:
7         r, c, dist = queue.popleft()
8         if is_target(r, c):
9             return dist
10        for dr, dc in directions:
11            nr, nc = r + dr, c + dc
12            if 0 ≤ nr < rows and 0 ≤ nc < cols and (nr, nc) not in visited:
13                if grid[nr][nc] ≠ WALL:
14                    visited.add((nr, nc))
15                    queue.append((nr, nc, dist + 1))
16    return -1

```

Template — Graph BFS

```

1 from collections import defaultdict, deque
2
3 def bfs_graph(graph, start):
4     visited = {start}
5     queue = deque([start])
6     order = []
7     while queue:
8         node = queue.popleft()
9         order.append(node)
10        for neighbor in graph[node]:
11            if neighbor not in visited:
12                visited.add(neighbor)
13                queue.append(neighbor)
14    return order

```

Complexity: Time $O(V + E)$, Space $O(V)$

Classic Problems

- › Binary Tree Level Order Traversal — LeetCode 102
- › Rotting Oranges — LeetCode 994
- › Number of Islands — LeetCode 200
- › Word Ladder — LeetCode 127

Pitfalls

- › Not marking visited when enqueueing (can enqueue same node multiple times → TLE or infinite loop).
- › Measuring `len(queue)` inside the inner loop (changes as you enqueue children).
- › Forgetting to handle disconnected graphs (multiple BFS starts or check all nodes).

3.8 DFS (Tree + Grid + Graph)

Trigger / When to Use

When you see: all paths, detect cycle, count components, flood fill, deep exploration before backtracking.

Core Idea Recursively (or with explicit stack) explore as far as possible down one branch before backtracking. Mark visited to prevent cycles.

Template — Tree DFS

```

1 def dfs_tree(node):
2     if not node:
3         return base_case
4     left = dfs_tree(node.left)
5     right = dfs_tree(node.right)
6     return combine(left, right, node.val)
7
8 # Iterative
9 def dfs_iterative(root):
10    stack = [root]
11    while stack:
12        node = stack.pop()
13        process(node)
14        if node.right: stack.append(node.right)
15        if node.left: stack.append(node.left)    # left last = processed first

```

Template — Grid DFS

```

1 def dfs_grid(grid, r, c, visited):
2     rows, cols = len(grid), len(grid[0])
3     if r < 0 or r ≥ rows or c < 0 or c ≥ cols:
4         return
5     if (r, c) in visited or grid[r][c] == 0:
6         return
7     visited.add((r, c))
8     for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
9         dfs_grid(grid, r + dr, c + dc, visited)

```

Template — Graph DFS / Cycle Detection

```

1 def has_cycle(graph, n):
2     WHITE, GRAY, BLACK = 0, 1, 2
3     color = [WHITE] * n
4
5     def dfs(u):
6         color[u] = GRAY
7         for v in graph[u]:
8             if color[v] == GRAY: # back edge = cycle
9                 return True
10            if color[v] == WHITE and dfs(v):
11                return True
12        color[u] = BLACK
13        return False
14
15    return any(color[u] == WHITE and dfs(u) for u in range(n))

```

Complexity: Time $O(V + E)$, Space $O(V)$ recursion stack

Classic Problems

- › Number of Islands — LeetCode 200
- › Max Area of Island — LeetCode 695
- › Path Sum II — LeetCode 113
- › Course Schedule — LeetCode 207

Pitfalls

- › Forgetting to mark visited before recursive call (infinite recursion on cycles).
- › Python recursion limit (~1000 default); use `sys.setrecursionlimit` or iterative DFS for large inputs.
- › Not restoring visited state when you need to explore all paths (use backtracking instead).

3.9 Backtracking

Trigger / When to Use

When you see: generate all valid configurations, constraint satisfaction (N-Queens, Sudoku), all paths in a graph.

Core Idea Build candidates incrementally. At each step, check if candidate is still valid. If not, prune and backtrack. If complete, record solution.

Template

```

1 def backtrack(candidates, result, current, start, *constraints):
2     if is_complete(current): # base case: valid solution
3         result.append(list(current))
4         return
5     for i in range(start, len(candidates)):
6         if not is_valid(current, candidates[i]): # prune
7             continue
8         current.append(candidates[i]) # choose
9         backtrack(candidates, result, current, i + 1, *constraints)
10        current.pop() # un-choose
11
12 # Concrete: Combination Sum (reuse allowed)
13 def combination_sum(candidates, target):

```

```

14     result = []
15     def backtrack(start, current, remaining):
16         if remaining == 0:
17             result.append(list(current))
18             return
19         if remaining < 0:
20             return
21         for i in range(start, len(candidates)):
22             current.append(candidates[i])
23             backtrack(i, current, remaining - candidates[i]) # i (not i+1): reuse ok
24             current.pop()
25     backtrack(0, [], target)
26     return result

```

Complexity: Time $O(2^N)$ or $O(N!)$ depending on branching factor; Space $O(N)$ recursion depth

Classic Problems

- › Combination Sum — LeetCode 39
- › N-Queens — LeetCode 51
- › Sudoku Solver — LeetCode 37
- › Word Search — LeetCode 79

Pitfalls

- › Not making a copy when appending to results (`result.append(current[:])`).
- › Passing `i+1` when reuse is allowed (should pass `i`).
- › Missing pruning conditions → TLE on large inputs.

3.10 Subsets / Permutations / Combinations

Trigger / When to Use

When you see: all subsets, power set, all permutations, all combinations of size K.

Core Idea For subsets: at each element, decide include or exclude. For permutations: at each position, try all unused elements. Backtracking framework handles both.

Template — Subsets

```

1 def subsets(nums):
2     result = []
3     def backtrack(start, current):
4         result.append(list(current)) # every prefix is a subset
5         for i in range(start, len(nums)):
6             current.append(nums[i])
7             backtrack(i + 1, current)
8             current.pop()
9     backtrack(0, [])
10    return result
11
12 # Bit manipulation alternative (elegant, no recursion)
13 def subsets_bit(nums):
14     n = len(nums)
15     return [[nums[j] for j in range(n) if (i >> j) & 1] for i in range(1 << n)]

```

Template – Permutations

```

1 def permutations(nums):
2     result = []
3     def backtrack(current, remaining):
4         if not remaining:
5             result.append(list(current))
6             return
7         for i in range(len(remaining)):
8             current.append(remaining[i])
9             backtrack(current, remaining[:i] + remaining[i+1:])
10            current.pop()
11    backtrack([], nums)
12    return result
13
14 # In-place swap permutations (more efficient)
15 def permutations_swap(nums):
16     result = []
17     def backtrack(start):
18         if start == len(nums):
19             result.append(nums[:])
20             return
21         for i in range(start, len(nums)):
22             nums[start], nums[i] = nums[i], nums[start]
23             backtrack(start + 1)
24             nums[start], nums[i] = nums[i], nums[start]
25    backtrack(0)
26    return result

```

Complexity: Subsets $O(2^N * N)$, Permutations $O(N! * N)$, Space $O(N)$

Classic Problems

- › Subsets – LeetCode 78
- › Subsets II (with duplicates) – LeetCode 90
- › Permutations – LeetCode 46
- › Combinations – LeetCode 77

Pitfalls

- › Subsets II: sort first, then skip $nums[i] == nums[i-1]$ at the same recursion level.
- › Permutations II: use a `used[]` array or sort + skip duplicates.
- › Forgetting `list(current)` copy when appending.

3.11 Binary Search (+ On Answer / Monotonic Predicate)

Trigger / When to Use

When you see: sorted array, rotated sorted array, "find minimum/maximum satisfying a condition", search space is monotonic, minimize the maximum / maximize the minimum.

Core Idea Halve the search space each iteration by comparing the midpoint. "Binary search on answer": define a predicate `feasible(x)` that is monotonically True/False over the answer space, then binary search on that space.

Template – Classic


```

1 def binary_search(arr, target):
2     left, right = 0, len(arr) - 1
3     while left ≤ right:
4         mid = left + (right - left) // 2    # avoids overflow
5         if arr[mid] == target:
6             return mid
7         elif arr[mid] < target:
8             left = mid + 1
9         else:
10            right = mid - 1
11    return -1
12
13 # Find leftmost occurrence
14 def lower_bound(arr, target):
15     left, right = 0, len(arr)
16     while left < right:
17         mid = (left + right) // 2
18         if arr[mid] < target:
19             left = mid + 1
20         else:
21             right = mid
22    return left

```

Template — On Answer

```

1 def binary_search_on_answer(lo, hi, feasible):
2     """Find the smallest x in [lo, hi] where feasible(x) is True."""
3     while lo < hi:
4         mid = (lo + hi) // 2
5         if feasible(mid):
6             hi = mid    # mid could be the answer, keep it
7         else:
8             lo = mid + 1
9     return lo
10
11 # Example: Koko Eating Bananas — LeetCode 875
12 def min_eating_speed(piles, h):
13     def can_finish(speed):
14         return sum((p + speed - 1) // speed for p in piles) ≤ h
15     return binary_search_on_answer(1, max(piles), can_finish)

```

Complexity: Time $O(\log N)$ classic, $O(\log(\text{answer_range}) * \text{verification_cost})$ on answer
Classic Problems

- › Binary Search — LeetCode 704
- › Search in Rotated Sorted Array — LeetCode 33
- › Koko Eating Bananas — LeetCode 875
- › Find Minimum in Rotated Sorted Array — LeetCode 153

Pitfalls

- › $\text{mid} = (\text{left} + \text{right}) // 2$ can overflow in other languages; use $\text{left} + (\text{right} - \text{left}) // 2$.
- › Off-by-one: $\text{left} \leq \text{right}$ vs $\text{left} < \text{right}$ — use $\leq$ for classic, $<$ for leftmost/rightmost variants.
- › Binary search on answer: define *feasible* correctly (monotonic), and set *hi* to a valid upper bound.

3.12 Top-K / Heap

Trigger / When to Use

When you see: top K largest, K smallest, K closest points, Kth largest/smallest element, K most frequent.

Core Idea Use a min-heap of size K for "top K largest" (smaller elements get evicted). Use a max-heap for "bottom K smallest". `heapq` in Python is a min-heap; negate values for max-heap.

Template

```

1 import heapq
2
3 # Top K largest elements - O(N log K)
4 def top_k_largest(nums, k):
5     min_heap = []
6     for num in nums:
7         heapq.heappush(min_heap, num)
8         if len(min_heap) > k:
9             heapq.heappop(min_heap) # evict smallest
10    return list(min_heap)
11
12 # Kth largest - O(N log K)
13 def kth_largest(nums, k):
14    return heapq.nlargest(k, nums)[-1] # or use the heap above
15
16 # K closest points to origin - O(N log K)
17 def k_closest(points, k):
18    return heapq.nsmallest(k, points, key=lambda p: p[0]**2 + p[1]**2)
19
20 # K most frequent - O(N log K)
21 from collections import Counter
22 def top_k_frequent(nums, k):
23    count = Counter(nums)
24    return heapq.nlargest(k, count.keys(), key=count.get)

```

Complexity: Time $O(N \log K)$, Space $O(K)$

Classic Problems

- › Kth Largest Element in an Array — LeetCode 215
- › K Closest Points to Origin — LeetCode 973
- › Top K Frequent Elements — LeetCode 347
- › Find K Pairs with Smallest Sums — LeetCode 373

Pitfalls

- › Python's `heapq` is min-heap only; negate for max-heap (`heapq.heappush(h, -val)`).
- › `heapq.nlargest(k, ...)` is $O(N \log K)$; fine for single calls but build the heap manually for streams.
- › For Kth largest, min-heap of size K is more memory-efficient than sorting.

3.13 K-way Merge

Trigger / When to Use

When you see: merge K sorted lists/arrays, find smallest range covering K lists, smallest in range.

Core Idea Use a min-heap with one element from each list. Pop the minimum, push the next element from that list. Repeat.

Template

```

1 import heapq
2
3 def merge_k_sorted_lists(lists):
4     """Merge K sorted linked lists into one sorted list."""
5     heap = []
6     for i, node in enumerate(lists):
7         if node:
8             heapq.heappush(heap, (node.val, i, node))
9     dummy = ListNode(0)
10    curr = dummy
11    while heap:
12        val, i, node = heapq.heappop(heap)
13        curr.next = node
14        curr = curr.next
15        if node.next:
16            heapq.heappush(heap, (node.next.val, i, node.next))
17    return dummy.next
18
19 def merge_k_sorted_arrays(arrays):
20     heap = [(arrays[i][0], i, 0) for i in range(len(arrays)) if arrays[i]]
21     heapq.heapify(heap)
22     result = []
23     while heap:
24         val, arr_idx, elem_idx = heapq.heappop(heap)
25         result.append(val)
26         if elem_idx + 1 < len(arrays[arr_idx]):
27             heapq.heappush(heap, (arrays[arr_idx][elem_idx+1], arr_idx, elem_idx+1))
28    return result

```

Complexity: Time $O(N \log K)$ where N = total elements, Space $O(K)$

Classic Problems

- › Merge K Sorted Lists — LeetCode 23
- › Find K Pairs with Smallest Sums — LeetCode 373
- › Smallest Range Covering Elements from K Lists — LeetCode 632
- › Kth Smallest Element in a Sorted Matrix — LeetCode 378

Pitfalls

- › Heap tuple must have unique tiebreakers if values can be equal (add index i to tuple).
- › Don't push None into the heap when a list is exhausted.

3.14 Two Heaps (Median of Stream)

Trigger / When to Use

When you see: median of a data stream, balance two halves, sliding window median.

Core Idea Maintain a max-heap for the lower half and a min-heap for the upper half. Always balance sizes so they differ by at most 1. Median is the top of the larger heap (or average of both tops).

Template

```

1 import heapq
2
3 class MedianFinder:
4     def __init__(self):
5         self.small = [] # max-heap (lower half): store negated values
6         self.large = [] # min-heap (upper half)
7
8     def add_num(self, num):
9         heapq.heappush(self.small, -num) # push to lower half
10        # Ensure max(small) ≤ min(large)
11        if self.small and self.large and (-self.small[0] > self.large[0]):
12            heapq.heappush(self.large, -heapq.heappop(self.small))
13        # Balance sizes
14        if len(self.small) > len(self.large) + 1:
15            heapq.heappush(self.large, -heapq.heappop(self.small))
16        elif len(self.large) > len(self.small):
17            heapq.heappush(self.small, -heapq.heappop(self.large))
18
19    def find_median(self):
20        if len(self.small) > len(self.large):
21            return -self.small[0]
22        return (-self.small[0] + self.large[0]) / 2.0

```

Complexity: Time $O(\log N)$ per insertion, $O(1)$ median query; Space $O(N)$

Classic Problems

- › Find Median from Data Stream — LeetCode 295
- › Sliding Window Median — LeetCode 480
- › IPO (maximize capital) — LeetCode 502

Pitfalls

- › Forgetting to negate when pushing to `small` (Python heap is min-heap).
- › Rebalancing order matters: first move invalid element, then balance sizes.
- › Sliding window median needs lazy deletion (mark removed elements, skip when popped).

3.15 Monotonic Stack

Trigger / When to Use

When you see: next greater/smaller element, previous greater/smaller, stock span, daily temperatures, largest rectangle in histogram, trapping rain water.

Core Idea Maintain a stack that is strictly increasing or decreasing. When a new element violates the order, pop elements (they've found their "next greater/smaller" answer).

Template — Next Greater Element

```

1 def next_greater_element(nums):
2     n = len(nums)
3     result = [-1] * n
4     stack = [] # stores indices; values are decreasing (monotonic decreasing)
5     for i in range(n):
6         while stack and nums[stack[-1]] < nums[i]:
7             idx = stack.pop()
8             result[idx] = nums[i] # nums[i] is the next greater for idx

```

```

9         stack.append(i)
10        return result
11
12    def daily_temperatures(temps):
13        result = [0] * len(temps)
14        stack = []
15        for i, t in enumerate(temps):
16            while stack and temps[stack[-1]] < t:
17                idx = stack.pop()
18                result[idx] = i - idx
19            stack.append(i)
20        return result
21
22    def largest_rectangle_histogram(heights):
23        stack = []
24        max_area = 0
25        heights.append(0)          # sentinel to flush stack
26        for i, h in enumerate(heights):
27            start = i
28            while stack and stack[-1][1] > h:
29                idx, height = stack.pop()
30                max_area = max(max_area, height * (i - idx))
31                start = idx
32            stack.append((start, h))
33        return max_area

```

Complexity: Time $O(N)$, Space $O(N)$

Classic Problems

- › Daily Temperatures — LeetCode 739
- › Next Greater Element I — LeetCode 496
- › Largest Rectangle in Histogram — LeetCode 84
- › Trapping Rain Water — LeetCode 42

Pitfalls

- › Deciding increasing vs decreasing: next greater → decreasing stack; next smaller → increasing stack.
- › Circular arrays: iterate $2*N$ with $i \% N$ indexing.
- › Histogram: store (start_index, height) not just height, to track span.

3.16 Prefix Sum

Trigger / When to Use

When you see: subarray sum equals K , range sum queries, count subarrays with given sum/property.

Core Idea $\text{prefix}[i]$ = sum of first i elements. Range sum $[l, r] = \text{prefix}[r+1] - \text{prefix}[l]$. For count of subarrays summing to K , use a hashmap of prefix sums seen so far.

Template

```

1 # Range sum query —  $O(1)$  per query after  $O(N)$  preprocessing
2 def build_prefix(nums):
3     prefix = [0] * (len(nums) + 1)

```

```

4     for i, num in enumerate(nums):
5         prefix[i+1] = prefix[i] + num
6     return prefix
7
8 def range_sum(prefix, l, r):          # inclusive [l, r]
9     return prefix[r+1] - prefix[l]
10
11 # Count subarrays with sum = k - O(N)
12 from collections import defaultdict
13 def subarray_sum_equals_k(nums, k):
14     count = 0
15     prefix = 0
16     seen = defaultdict(int)
17     seen[0] = 1                      # empty prefix
18     for num in nums:
19         prefix += num
20         count += seen[prefix - k]     # how many prefixes end at (prefix - k)
21         seen[prefix] += 1
22     return count

```

Complexity: Build $O(N)$, Query $O(1)$; Subarray count $O(N)$, Space $O(N)$

Classic Problems

- › Subarray Sum Equals K — LeetCode 560
- › Range Sum Query - Immutable — LeetCode 303
- › Count of Range Sum — LeetCode 327
- › Product of Array Except Self — LeetCode 238

Pitfalls

- › Initialize `seen[0] = 1` (the empty prefix with sum 0 exists once).
- › For 2D prefix sums, use inclusion-exclusion: `pre[r2][c2] - pre[r1-1][c2] - pre[r2][c1-1] + pre[r1-1][c1-1]`.
- › Negative numbers are fine; this works regardless.

3.17 Union-Find (DSU)

Trigger / When to Use

When you see: connected components, dynamic connectivity, cycle detection in undirected graph, redundant connection, accounts merge.

Core Idea Each element points to a parent. `find(x)` returns root of x's component (with path compression). `union(x, y)` merges components (with rank/size to keep tree flat).

Template

```

1 class UnionFind:
2     def __init__(self, n):
3         self.parent = list(range(n))
4         self.rank = [0] * n
5         self.components = n
6
7     def find(self, x):
8         if self.parent[x] != x:
9             self.parent[x] = self.find(self.parent[x]) # path compression

```

```

10     return self.parent[x]
11
12     def union(self, x, y):
13         px, py = self.find(x), self.find(y)
14         if px == py:
15             return False                    # already connected
16         if self.rank[px] < self.rank[py]:
17             px, py = py, px
18         self.parent[py] = px                # union by rank
19         if self.rank[px] == self.rank[py]:
20             self.rank[px] += 1
21         self.components -= 1
22         return True
23
24     def connected(self, x, y):
25         return self.find(x) == self.find(y)

```

Complexity: Near $O(1)$ amortized per operation (inverse Ackermann $\alpha(N)$); Space $O(N)$

Classic Problems

- › Number of Connected Components in Undirected Graph — LeetCode 323
- › Redundant Connection — LeetCode 684
- › Accounts Merge — LeetCode 721
- › Making a Large Island — LeetCode 827

Pitfalls

- › Not using both path compression AND union by rank (without both, worst case degrades to $O(N)$).
- › Forgetting to check if $\text{find}(x) = \text{find}(y)$ before union (cycle detection).
- › Off-by-one with node numbering (0-indexed vs 1-indexed).

3.18 Topological Sort

Trigger / When to Use

When you see: course prerequisites, task scheduling, build dependency order, DAG ordering, alien dictionary.

Core Idea Kahn's algorithm (BFS): compute in-degrees, start with 0 in-degree nodes, process each and reduce neighbors' in-degrees. **DFS-based:** finish time ordering — DFS, push to stack on exit, reverse the stack.

Template — Kahn's (BFS)

```

1  from collections import deque, defaultdict
2
3  def topo_sort_kahn(n, prerequisites):
4      graph = defaultdict(list)
5      in_degree = [0] * n
6      for a, b in prerequisites:          # b → a (b must come before a)
7          graph[b].append(a)
8          in_degree[a] += 1
9      queue = deque([i for i in range(n) if in_degree[i] == 0])
10     order = []

```

```

11     while queue:
12         node = queue.popleft()
13         order.append(node)
14         for neighbor in graph[node]:
15             in_degree[neighbor] -= 1
16             if in_degree[neighbor] == 0:
17                 queue.append(neighbor)
18     return order if len(order) == n else [] # empty = cycle exists

```

Template – DFS-based

```

1 def topo_sort_dfs(n, graph):
2     WHITE, GRAY, BLACK = 0, 1, 2
3     color = [WHITE] * n
4     result = []
5     has_cycle = [False]
6
7     def dfs(u):
8         if has_cycle[0]: return
9         color[u] = GRAY
10        for v in graph[u]:
11            if color[v] == GRAY:
12                has_cycle[0] = True; return
13            if color[v] == WHITE:
14                dfs(v)
15        color[u] = BLACK
16        result.append(u)
17
18    for u in range(n):
19        if color[u] == WHITE:
20            dfs(u)
21    return [] if has_cycle[0] else result[::-1]

```

Complexity: Time $O(V + E)$, Space $O(V + E)$

Classic Problems

- › Course Schedule – LeetCode 207
- › Course Schedule II – LeetCode 210
- › Alien Dictionary – LeetCode 269
- › Sequence Reconstruction – LeetCode 444

Pitfalls

- › Cycle = no valid ordering: `len(order) == n` check in Kahn's detects it.
- › Building graph in wrong direction (which node blocks which).
- › Alien Dictionary: adjacent characters in each word define edges; must check invalid input (longer word before shorter prefix → return "").

3.19 Trie

Trigger / When to Use

When you see: word search, prefix matching, autocomplete, count words with prefix, replace words with shortest root.

Core Idea A tree where each node represents a character. Each root-to-leaf path spells a

word. Supports $O(L)$ insert/search (L = word length) regardless of dictionary size.

Template

```

1 class TrieNode:
2     def __init__(self):
3         self.children = {}
4         self.is_end = False
5
6 class Trie:
7     def __init__(self):
8         self.root = TrieNode()
9
10    def insert(self, word):
11        node = self.root
12        for ch in word:
13            if ch not in node.children:
14                node.children[ch] = TrieNode()
15            node = node.children[ch]
16        node.is_end = True
17
18    def search(self, word):
19        node = self._find_node(word)
20        return node is not None and node.is_end
21
22    def starts_with(self, prefix):
23        return self._find_node(prefix) is not None
24
25    def _find_node(self, prefix):
26        node = self.root
27        for ch in prefix:
28            if ch not in node.children:
29                return None
30            node = node.children[ch]
31        return node

```

Complexity: Time $O(L)$ per operation (L = word length); Space $O(\text{total characters} * \text{alphabet size})$

Classic Problems

- › Implement Trie (Prefix Tree) — LeetCode 208
- › Word Search II — LeetCode 212
- › Replace Words — LeetCode 648
- › Design Search Autocomplete System — LeetCode 642

Pitfalls

- › Using a dict vs array for children: dict is flexible; `[None]*26` is faster for lowercase English only.
- › search vs starts_with: `is_end` flag differentiates them.
- › Word Search II: prune the Trie as words are found to avoid revisiting.

3.20 Dynamic Programming

Trigger / When to Use

When you see: count ways, min/max cost/steps, optimal value, overlapping subproblems (same sub-calculation repeated), optimal substructure (optimal solution uses optimal

sub-solutions).

Core Idea Break problem into subproblems; store results (memoization/tabulation). Key: define $dp[i]$ or $dp[i][j]$ precisely, write the recurrence, identify base cases.

DP --- 0/1 Knapsack

Trigger: "pick or skip" each item, capacity constraint, maximize/count value.

```
1 def knapsack_01(weights, values, capacity):
2     n = len(weights)
3     # dp[i][w] = max value using first i items with capacity w
4     dp = [[0] * (capacity + 1) for _ in range(n + 1)]
5     for i in range(1, n + 1):
6         for w in range(capacity + 1):
7             # skip item i
8             dp[i][w] = dp[i-1][w]
9             # take item i (if it fits)
10            if weights[i-1] ≤ w:
11                dp[i][w] = max(dp[i][w], dp[i-1][w - weights[i-1]] + values[i-1])
12    return dp[n][capacity]
13
14 # Space-optimized to 1D (iterate w in reverse!)
15 def knapsack_01_1d(weights, values, capacity):
16     dp = [0] * (capacity + 1)
17     for i in range(len(weights)):
18         for w in range(capacity, weights[i] - 1, -1): # REVERSE to avoid reuse
19             dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
20    return dp[capacity]
```

Classic: 0/1 Knapsack, Subset Sum (LeetCode 416), Partition Equal Subset Sum (LeetCode 416), Target Sum (LeetCode 494).

DP --- Unbounded Knapsack

Trigger: "unlimited copies" of each item, coin change (any number of coins).

```
1 def unbounded_knapsack(weights, values, capacity):
2     dp = [0] * (capacity + 1)
3     for w in range(1, capacity + 1):
4         for i in range(len(weights)):
5             if weights[i] ≤ w:
6                 dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
7     return dp[capacity]
8
9 def coin_change(coins, amount):
10    dp = [float('inf')] * (amount + 1)
11    dp[0] = 0
12    for a in range(1, amount + 1):
13        for coin in coins:
14            if coin ≤ a:
15                dp[a] = min(dp[a], dp[a - coin] + 1)
16    return dp[amount] if dp[amount] ≠ float('inf') else -1
```

Classic: Coin Change (LeetCode 322), Coin Change II (LeetCode 518), Rod Cutting.
Key difference from 0/1: iterate w forward (allows reuse of same item).

DP --- LCS (Longest Common Subsequence)

Trigger: two strings/sequences, longest common subsequence, edit distance, shortest common supersequence.

```

1 def lcs(s1, s2):
2     m, n = len(s1), len(s2)
3     dp = [[0] * (n + 1) for _ in range(m + 1)]
4     for i in range(1, m + 1):
5         for j in range(1, n + 1):
6             if s1[i-1] == s2[j-1]:
7                 dp[i][j] = dp[i-1][j-1] + 1
8             else:
9                 dp[i][j] = max(dp[i-1][j], dp[i][j-1])
10    return dp[m][n]
11
12 def edit_distance(word1, word2):
13     m, n = len(word1), len(word2)
14     dp = [[0] * (n + 1) for _ in range(m + 1)]
15     for i in range(m + 1): dp[i][0] = i
16     for j in range(n + 1): dp[0][j] = j
17     for i in range(1, m + 1):
18         for j in range(1, n + 1):
19             if word1[i-1] == word2[j-1]:
20                 dp[i][j] = dp[i-1][j-1]
21             else:
22                 dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
23    return dp[m][n]

```

Classic: LCS (LeetCode 1143), Edit Distance (LeetCode 72), Shortest Common Supersequence (LeetCode 1092).

DP --- LIS (Longest Increasing Subsequence)

Trigger: longest strictly increasing subsequence, patience sorting, number of LIS.

```

1 def lis_dp(nums):
2     """O(N^2) DP"""
3     n = len(nums)
4     dp = [1] * n
5     for i in range(1, n):
6         for j in range(i):
7             if nums[j] < nums[i]:
8                 dp[i] = max(dp[i], dp[j] + 1)
9     return max(dp)
10
11 def lis_binary_search(nums):
12     """O(N log N) using patience sorting"""
13     from bisect import bisect_left
14     tails = []
15     for num in nums:
16         pos = bisect_left(tails, num)
17         if pos == len(tails):
18             tails.append(num)
19         else:
20             tails[pos] = num
21     return len(tails)

```

Classic: Longest Increasing Subsequence (LeetCode 300), Russian Doll Envelopes (Leet-

Code 354), Number of LIS (LeetCode 673).

DP --- Matrix / Grid DP

Trigger: grid, min path sum, unique paths, count paths, max square of 1s.

```

1 def min_path_sum(grid):
2     m, n = len(grid), len(grid[0])
3     dp = [[0]*n for _ in range(m)]
4     dp[0][0] = grid[0][0]
5     for i in range(1, m): dp[i][0] = dp[i-1][0] + grid[i][0]
6     for j in range(1, n): dp[0][j] = dp[0][j-1] + grid[0][j]
7     for i in range(1, m):
8         for j in range(1, n):
9             dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])
10    return dp[m-1][n-1]
11
12 def maximal_square(matrix):
13     if not matrix: return 0
14     m, n = len(matrix), len(matrix[0])
15     dp = [[0]*n for _ in range(m)]
16     max_side = 0
17     for i in range(m):
18         for j in range(n):
19             if matrix[i][j] == '1':
20                 if i == 0 or j == 0:
21                     dp[i][j] = 1
22                 else:
23                     dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1
24                 max_side = max(max_side, dp[i][j])
25    return max_side * max_side

```

Classic: Unique Paths (LeetCode 62), Minimum Path Sum (LeetCode 64), Maximal Square (LeetCode 221), Dungeon Game (LeetCode 174).

DP --- Interval DP

Trigger: merge stones, burst balloons, matrix chain multiplication, palindrome partitioning.

```

1 def burst_balloons(nums):
2     """LeetCode 312 - classic interval DP"""
3     nums = [1] + nums + [1]
4     n = len(nums)
5     dp = [[0]*n for _ in range(n)]
6     for length in range(2, n): # interval length
7         for left in range(n - length):
8             right = left + length
9             for k in range(left+1, right): # k = last balloon to burst in (left, right)
10                dp[left][right] = max(
11                    dp[left][right],
12                    dp[left][k] + nums[left]*nums[k]*nums[right] + dp[k][right]
13                )
14    return dp[0][n-1]

```

Classic: Burst Balloons (LeetCode 312), Minimum Cost to Merge Stones (LeetCode 1000), Strange Printer (LeetCode 664).

DP --- Subsequences / Strings

Trigger: count distinct subsequences, palindromic subsequences, word break.

```

1 def num_distinct_subsequences(s, t):
2     """Count distinct subsequences of s equal to t - LeetCode 115"""
3     m, n = len(s), len(t)
4     dp = [[0]*(n+1) for _ in range(m+1)]
5     for i in range(m+1): dp[i][0] = 1 # empty t matched
6     for i in range(1, m+1):
7         for j in range(1, n+1):
8             dp[i][j] = dp[i-1][j] # don't use s[i-1]
9             if s[i-1] == t[j-1]:
10                 dp[i][j] += dp[i-1][j-1] # use s[i-1]
11     return dp[m][n]
12
13 def word_break(s, word_dict):
14     """LeetCode 139"""
15     word_set = set(word_dict)
16     n = len(s)
17     dp = [False] * (n + 1)
18     dp[0] = True
19     for i in range(1, n+1):
20         for j in range(i):
21             if dp[j] and s[j:i] in word_set:
22                 dp[i] = True; break
23     return dp[n]

```

Classic: Distinct Subsequences (LeetCode 115), Word Break (LeetCode 139), Palindromic Substrings (LeetCode 647).

3.21 Greedy

Trigger / When to Use

When you see: "minimum number of X", "maximize Y", local decisions that don't affect future options, exchange argument, interval scheduling.

Core Idea Make the locally optimal choice at each step. Prove (or intuit) that no better global solution exists. Common proof technique: "exchange argument" — show swapping any element for the greedy choice can only improve or maintain the solution.

Template — Interval Scheduling Maximization

```

1 def max_non_overlapping_intervals(intervals):
2     """Maximum number of non-overlapping intervals (greedy: earliest end first)"""
3     intervals.sort(key=lambda x: x[1]) # sort by END time
4     count = 0
5     last_end = float('-inf')
6     for start, end in intervals:
7         if start >= last_end:
8             count += 1
9             last_end = end
10    return count
11
12 def jump_game(nums):
13     """Can reach last index? - LeetCode 55"""

```

```

14     max_reach = 0
15     for i, num in enumerate(nums):
16         if i > max_reach:
17             return False
18         max_reach = max(max_reach, i + num)
19     return True
20
21 def jump_game_ii(nums):
22     """Minimum jumps to reach last index - LeetCode 45"""
23     jumps = cur_end = cur_far = 0
24     for i in range(len(nums) - 1):
25         cur_far = max(cur_far, i + nums[i])
26         if i == cur_end:
27             jumps += 1
28             cur_end = cur_far
29     return jumps

```

Complexity: Usually $O(N \log N)$ (sort) + $O(N)$; Space $O(1)$

Classic Problems

- › Jump Game — LeetCode 55
- › Non-overlapping Intervals — LeetCode 435
- › Task Scheduler — LeetCode 621
- › Gas Station — LeetCode 134

Pitfalls

- › Not all greedy approaches are correct; verify with a counterexample.
- › Activity selection: sort by END (not start, not duration).
- › When greedy fails, fall back to DP.

3.22 Bit Manipulation

Trigger / When to Use

When you see: find unique element in array where all others appear twice/three times, count set bits, power of 2, XOR tricks, subset enumeration.

Core Idea Exploit binary properties: XOR cancels duplicates, AND/OR extract/set bits, shifting multiplies/divides by powers of 2.

Key Tricks

```

1 # XOR: a ^ a = 0, a ^ 0 = a
2 def single_number(nums):
3     """Find the element appearing once - LeetCode 136"""
4     result = 0
5     for num in nums:
6         result ^= num
7     return result
8
9 # Count set bits (Brian Kernighan)
10 def count_bits(n):
11     count = 0
12     while n:
13         n &= (n - 1)    # removes lowest set bit

```

```

14         count += 1
15     return count
16
17 # Check power of 2
18 def is_power_of_two(n):
19     return n > 0 and (n & (n - 1)) == 0
20
21 # Get / Set / Clear bit
22 def get_bit(n, i): return (n >> i) & 1
23 def set_bit(n, i): return n | (1 << i)
24 def clear_bit(n, i): return n & ~(1 << i)
25
26 # Enumerate all subsets of a bitmask
27 def enumerate_subsets(mask):
28     sub = mask
29     while sub:
30         process(sub)
31         sub = (sub - 1) & mask # next smaller submask
32
33 # XOR to find two unique numbers (all others appear twice)
34 def single_number_iii(nums):
35     """LeetCode 260"""
36     xor = 0
37     for n in nums: xor ^= n # xor of the two unique numbers
38     diff_bit = xor & (-xor) # rightmost set bit
39     a = b = 0
40     for n in nums:
41         if n & diff_bit: a ^= n
42         else: b ^= n
43     return [a, b]

```

Complexity: $O(N)$ time, $O(1)$ space for most tricks

Classic Problems

- › Single Number — LeetCode 136
- › Number of 1 Bits — LeetCode 191
- › Counting Bits — LeetCode 338
- › Reverse Bits — LeetCode 190
- › Single Number III — LeetCode 260

Pitfalls

- › Python integers have arbitrary precision; no overflow, but be careful with $\sim n$ (gives $-n - 1$).
- › Use $n \& (-n)$ to isolate the lowest set bit.
- › Bitmask DP: remember $1 \ll n$ can be large; check constraints.

4 Complexity Quick Reference

4.1 Big-O of Common Data Structure Operations

Operation	Array	Linked List	Hash Map	BST (avg)	Heap	Sorted Array
Access by index	$O(1)$	$O(N)$	N/A	N/A	N/A	$O(1)$
Search	$O(N)$	$O(N)$	$O(1)$	$O(\log N)$	$O(N)$	$O(\log N)$
Insert (end)	$O(1)^*$	$O(1)$	$O(1)$	$O(\log N)$	$O(\log N)$	$O(N)$

Operation	Array	Linked List	Hash Map	BST (avg)	Heap	Sorted Array
Insert (middle)	$O(N)$	$O(1)^{**}$	$O(1)$	$O(\log N)$	$O(\log N)$	$O(N)$
Delete	$O(N)$	$O(1)^{**}$	$O(1)$	$O(\log N)$	$O(\log N)$	$O(N)$
Min/Max	$O(N)$	$O(N)$	$O(N)$	$O(\log N)$	$O(1)$	$O(1)$

*Amortized for dynamic array. **Given a pointer to the node.

4.2 Common Algorithm Complexities

Algorithm	Time	Space
Binary Search	$O(\log N)$	$O(1)$
Quick Sort	$O(N \log N)$ avg, $O(N^2)$ worst	$O(\log N)$
Merge Sort	$O(N \log N)$	$O(N)$
Heap Sort	$O(N \log N)$	$O(1)$
BFS / DFS	$O(V + E)$	$O(V)$
Dijkstra (heap)	$O((V+E) \log V)$	$O(V)$
Topological Sort	$O(V + E)$	$O(V)$
Union-Find	$O(\alpha(N))$ per op	$O(N)$
Trie insert/search	$O(L)$	$O(L)$

4.3 Acceptable Complexity vs Input Size N

N (input size)	Max acceptable complexity	Notes
$N \leq 10$	$O(N!)$	Permutations, backtracking with full exploration
$N \leq 20$	$O(2^N)$	Bitmask DP, subset enumeration
$N \leq 100$	$O(N^3)$	Matrix chain, interval DP, Floyd-Warshall
$N \leq 500$	$O(N^2)$	Some DP, all-pairs work
$N \leq 1,000$	$O(N^2)$	Bubble/insertion sort if needed, $O(N^2)$ DP
$N \leq 10,000$	$O(N^2 \log N)$	Rarely; $O(N^2)$ should work
$N \leq 100,000$	$O(N \log N)$	Sort, heap operations, segment tree
$N \leq 1,000,000$	$O(N \log N)$	Must be near-linear
$N \leq 10,000,000$	$O(N)$	Linear only – prefix sum, two pointers, sliding window
$N > 10^8$	$O(\log N)$ or $O(1)$	Binary search, math tricks

Rule: Assume $\sim 10^8$ simple operations per second for Python (10^9 for C++). Multiply complexity by constant factors; Python is ~ 5 - 10 x slower than C++.

5 Interview Communication Tips

5.1 Before You Code

1. **Repeat the problem** in your own words. "So we're given... and we need to return..."
2. **Clarify edge cases** out loud: empty input? negative numbers? duplicates allowed? integer overflow?
3. **State constraints** you noticed: "N can be up to 10^5 , so we need $O(N \log N)$ or better."
4. **Start with brute force** — even if obvious: "The naive approach is $O(N^2)$ by checking all pairs. We can do better..."
5. **Think aloud** while optimizing: "I notice the array is sorted, so maybe we can binary search... or use two pointers..."

5.2 While Coding

- › **Name variables clearly:** left, right, window_sum, not i, j, temp.
- › **Comment key invariants:** # left: next position to fill, # window is always valid here.
- › **Write helper functions** for repetitive logic rather than duplicating code.
- › **Don't go silent** for more than 30 seconds; narrate your thought process.

5.3 After Coding

1. **Trace through your example** step by step on the code you wrote.
2. **Check edge cases:** empty array, single element, all same, already sorted, negative numbers.
3. **State complexity** proactively: "Time is $O(N \log N)$ for the sort plus $O(N)$ for the scan, so $O(N \log N)$. Space is $O(1)$ if we ignore the output."
4. **Ask if they want you to optimize** before jumping to a different approach.

5.4 Recovery Tactics

- › Stuck? Say "Let me think about what information I'm not using yet."
- › Hint? "That's helpful — so if we track X, we could avoid recomputing Y..."
- › Bug? "Let me re-trace this test case to find where it diverges."

6 Typical Mistakes Candidates Make

Mistake	Fix
Jumping to code without clarifying	Always spend 2-3 min clarifying before touching code
Skipping brute force	State it first; it demonstrates structured thinking and sets a baseline
Not handling edge cases	Build a checklist: empty, single element, all same, negatives, overflow
Vague variable names (i, j, tmp)	Use descriptive names even if it takes 5 more seconds
Not stating complexity	Interviewers almost always ask; volunteer it proactively
Silently debugging	Narrate: "I think the bug is here because..."
Confusing inclusive/exclusive bounds	Write # inclusive OR # exclusive in comments
Off-by-one in binary search	Stick to one template and test [1], [1, 2], [1, 2, 3]
Forgetting to copy list before appending	result.append(current[:]) NOT result.append(current)
Not tracking the visited set	Add it before BFS/DFS; mark visited when enqueueing
Wrong loop order in DP	0/1 knapsack: reverse; unbounded: forward

Mistake	Fix
Greedy without justification	Briefly argue why greedy works or think about exchange argument
Python recursion limit	Add <code>sys.setrecursionlimit(10**6)</code> for deep DFS
Mutating input	Ask "can I modify the input?" or work on a copy

7 Revision Cheat Sheet

7.1 The 20 Must-Know Patterns

#	Pattern	1-line Memory Hook
1	Two Pointers	Sorted + pair/triplet → shrink from both ends
2	Sliding Window	Contiguous subarray condition → shrink/grow window
3	Fast & Slow Pointers	Cycle / middle → tortoise and hare
4	Merge Intervals	Overlapping intervals → sort by start, merge greedily
5	Cyclic Sort	Array with $[1..N]$ → each number belongs at index $n-1$
6	LL Reversal	Reverse linked list → prev/curr/next dance
7	BFS	Shortest path / level order → queue + visited
8	DFS / Backtracking	All paths / all configs → recurse + undo
9	Binary Search	Sorted / monotonic predicate → halve search space
10	Top-K Heap	K largest/smallest → min-heap of size K
11	K-way Merge	K sorted lists → heap with one from each
12	Two Heaps	Median of stream → max-heap left, min-heap right
13	Monotonic Stack	Next greater/smaller → pop when violated
14	Prefix Sum	Subarray sum = K → hash map of prefix sums
15	Union-Find	Dynamic connectivity → compress + rank
16	Topo Sort	DAG ordering / cycle → Kahn's (BFS in-degree)
17	Trie	Prefix search → character tree
18	0/1 Knapsack DP	Pick or skip + capacity → 2D or reverse-1D DP
19	Greedy	Local optimal = global optimal → sort + scan
20	Bit Manipulation	XOR / AND / shifts → bitmask tricks

7.2 Decision Tree (30-second pattern picker)

```

Is array sorted or can we sort it?
├─ Yes → Two Pointers or Binary Search
└─ No
    ├─ Subarray/substring condition? → Sliding Window
    ├─ Linked list? → Fast & Slow or LL Reversal
    └─ Graph/tree?
        ├─ Shortest path / min steps? → BFS
        ├─ All paths / components? → DFS / Backtracking
        ├─ Dependency order? → Topological Sort
        └─ Connected components (dynamic)? → Union-Find
    ├─ K elements? → Heap (Top-K or K-way Merge)
    ├─ Intervals? → Merge Intervals
    ├─ Count ways / optimal value? → Dynamic Programming
    └─ Prefix search / autocomplete? → Trie
  
```

- └─ Next greater / histogram? → Monotonic Stack
- └─ Subarray sum queries? → Prefix Sum
- └─ XOR / bits / unique? → Bit Manipulation

7.3 Quick Complexity Reminder

- $O(1)$ → hash map lookup, heap top
- $O(\log N)$ → binary search, heap push/pop
- $O(N)$ → single pass, two pointers, sliding window
- $O(N \log N)$ → sort, N heap operations
- $O(N^2)$ → nested loops, some DP
- $O(2^N)$ → subsets, bitmask DP
- $O(N!)$ → permutations

Last updated: 2026-06-14 | Covers ~25 patterns with 100+ LeetCode problems